

there will be an *index.html* file, a *config.xml* file and the *icon.png* file. Then there will be the respective folders for the library files (containing the *angular.js* file), the *jquery-mobile* files, the images to be used, the custom JavaScript code, and the custom styling, if needed.

Let us first look into the relevant code snippet from the *index.html* file:

```
<div data-role="page">

<div data-role="header">
<h1>Cars</h1>
</div>

<div data-role="content" id="main" ng-app="carsApp" ng-
controller="myAppController">
<a data-role="button" ng-click="fetchCarMakes()">Get Makes</
a>

<ul data-role="listview" data-inset="true"
id="dataListView">
<li ng-repeat="data in listViewData">
<a ng-click="fetchModels(data)">

{{data.name}}
</a>
</li>
</ul>
</div>

</div>
```

Here, we are creating a page using jQuery Mobile. To convert any *div* tag into a page in jQuery Mobile, we just need to add *data-role="page"* in the *div* tag. The page has two major sets of content—the page header and the page content—each declared using its respective *data-role*. The content of the page has one button, which when clicked by the user can get the names of all the car companies from around the world. This data is displayed in the *listview* below the button. The *content* *div* uses two AngularJS declarations—*ng-app* and *ng-controller*. The first one allows the developer to declare a root element for the application whose behaviour can be modified. *ng-controller* lets the developer specify a JavaScript controller class that can be used to evaluate HTML expressions. The *li* element is the *listview* using the *ng-repeat* declaration. *ng-repeat* helps the user to instantiate elements by using items from a collection. Here, the collection is the *listViewData*, which will be declared inside the controller class—*myAppController*. The image source can use *ng-src*. All the data that needs to be accessed in the HTML can be accessed using *{{}}*.

Next, let us look at the relevant snippet from the JavaScript:

```
angular.module('carsApp', [])
.controller('myAppController', ['$scope', '$http',
function($scope,$http) {
    $scope.listViewData = [];
    $scope.fetchCarMakes = function() {
    $.mobile.loading( "show" );
    $scope.listViewData = [];
    $http({method: 'GET', url: getCarMakesURL}).
    success(function(data, status, headers, config) {
        angular.forEach(data, function(value,obj){
            angular.forEach(value, function(makesDetails,ind
ex){
                switch (makesDetails.make_country) {
                    case "Italy":
                        $scope.listViewData.push({name:makesDetails.
make_display,countryFlag: "italy.png",id:makesDetails.make_
id});
                        break;
                        //..
                        //..
                        //..
                        $("#dataListView").
                        listview("refresh");
                        //..
                        //..
                    }
                $scope.fetchModels = function(makeName){
                    //..
                    $http({method: 'GET', url:modelMakeURL}).
                    success(function(data, status, headers, config) {
                        angular.forEach(data, function(value,obj){
                            angular.forEach(value, function(modelDetails,ind
ex){
                                $scope.listViewData.push({name:modelDetails.
model_name,countryFlag: makeName.countryFlag,id:
modelDetails.model_make_id});
                            });
                        });
                        //..
                        //..
                        //..
                    }
                })
            })
        })
    })
}
```

Here, for *ng-app="carsApp"* declared in the HTML, we declare an angular module, which contains our controller. Inside the controller, we handle two events—the button click and the *listview* item click.

There are two major objects used in the controller - *\$scope* and *\$http*. The *\$scope* object refers to the application model. Scope is the glue between the application controller and the view. Hence, using Scope, we can directly make changes in the view with our controller. The *\$http* object helps for communication with remote HTTP servers.